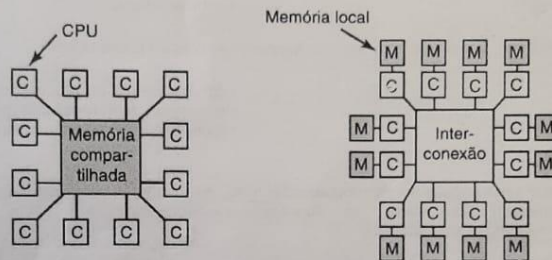




Universidade de Caxias do Sul
Área do Conhecimento de Ciências Exatas e Engenharias
CIC4004A - Programação Concorrente, Paralela e Distribuída
Professor: André Luis Martinotto
Primeira Avaliação

QUESTÃO 1) (1,5 PONTOS) Na figura abaixo, retirada do livro "Sistemas Operacionais Modernos" de Andrew Tanenbaum, tem-se diferentes exemplos de arquiteturas paralelas.

- Identifique a qual categoria da taxonomia de Flynn pertencem as arquiteturas abaixo. Justifique sua resposta.
- Quais dessas arquiteturas paralelas podem ser classificadas com um sistema distribuído? Por quê?



QUESTÃO 2) (1,0 PONTOS) Considere uma situação em que sensores de IoT capturam dados de temperatura em tempo real e enviam para um servidor que controla a temperatura de um armazém. Se os dados não chegarem ao servidor dentro de um limite de tempo, eles são descartados para evitar informações desatualizadas na gestão de estoques sensíveis à temperatura. Qual protocolo de comunicação, TCP ou UDP, seria mais apropriado para garantir a eficácia na transmissão dos dados e atender às necessidades específicas dessa aplicação? Justifique a sua escolha com base nas características dos dois protocolos.

QUESTÃO 3) (1,0 PONTO) Protocol Buffers (Protobuf) é um formato de dados binário, enquanto o JSON (JavaScript Object Notation) é um formato textual. Ambos são comumente empregados na comunicação em sistemas distribuídos. Ao analisar esses formatos, compare as características de Protocol Buffers e JSON. Quais são as vantagens de cada um e de que forma essa escolha pode impactar o desempenho em ambientes distribuídos?

QUESTÃO 4) (1,5 PONTO) Um programa que realiza um processamento intensivo leva 100 segundos para ser executado em um único núcleo de CPU. Após a implementação de uma versão paralela, o mesmo programa é executado em 4 núcleos de CPU e leva 30 segundos para ser concluído.

- Calcule o speedup obtido com a execução do programa em paralelo.
- Calcule a eficiência do programa em relação ao número de núcleos utilizados.

QUESTÃO 5) (2,5 PONTOS) Você foi encarregado de desenvolver uma aplicação cliente/servidor utilizando sockets (TCP) para transferência de arquivos. Nessa aplicação o processo cliente lê o nome de um arquivo e faz uma requisição desse arquivo ao processo servidor. O processo servidor recebe essa requisição e efetua a transferência do arquivo ao processo cliente.

Deseja-se que o servidor possua a capacidade de atender múltiplos clientes simultaneamente, isto é, no caso de uma requisição de um cliente por um arquivo grande os demais clientes não deverão ficar esperando para serem atendidos.

Para o desenvolvimento da aplicação você deve utilizar uma biblioteca de sockets que apresenta as seguintes primitivas:

<code>sock = socket(Protocolo):bind(socket, porta):</code>	Criação de um <code>socket</code> . Protocolo TCP ou UDP.
<code>bind(socket, porta):</code>	Estabelece uma porta ao <code>socket</code> .
<code>listen(socket, quantos):</code>	Definição do tamanho da lista de requisições.
<code>connect (socket, ip, porta):</code>	Requisição de uma conexão.
<code>new_sock = accept (socket):</code>	Espera por uma conexão. Retorna um <code>socket</code> que será usado para a comunicação.
<code>send (socket, dados):</code>	Envia uma mensagem com o conteúdo de dados;
<code>recv(socket, dados):</code>	Recebe uma mensagem e armazena em dados;
<code>close(socket):</code>	Fecha a conexão;

Para permitir o atendimento de múltiplos clientes simultaneamente utilize a primitiva `fork`:

<code>int fork():</code>	cria uma cópia do processo original. A chamada de sistema <code>fork</code> retorna o <code>pid</code> do filho para o processo pai e 0 para o processo filho.
--------------------------	--

OBSERVAÇÃO: Utilize uma linguagem de programação real. As funções para transferência de arquivos não precisam ser implementadas utilize as funções `enviarArquivo(socket, string nomeArquivo)` e `receberArquivo(socket, string nomeArquivo)`.

↳ envia um arquivo. Para, requisição do cliente e com resposta

QUESTÃO 6) (2,5 PONTOS) O trecho de código abaixo calcula a variância e o desvio padrão para um vetor `amostra[N]`. *→ iterar em todos os processos*

```
soma = 0;
for (i=0; i<N; i++){
    soma += amostra[i];
}
media = soma/N;

soma = 0;
for (i=0; i<N; i++){
    soma += (amostra[i]-media) * (amostra[i]-media);
}

variancia = soma/(N-1);
desvio = sqrt(variancia);
```

Utilizando como suporte uma biblioteca de troca de mensagens que segue o modelo SPMD (único programa trabalhando sobre múltiplos dados) e que possua as primitivas abaixo e uma linguagem de programação real, reescreva o programa para que o cálculo seja efetuado em paralelo por P processos.

Primitivas disponíveis:

<code>int rank():</code>	Retorna o identificador de cada processo (de 0 a $N-1$);
<code>int size():</code>	Retorna o número de processos;
<code>send(<var>, <id>):</code>	Envia uma mensagem contida em <code><var></code> para a tarefa identificada por <code><id></code> .
<code>receive(<var>, <id>):</code>	Recebe do processo <code><id></code> e armazena no endereço <code><var></code> ;

1) a - Ambas são MIMD, isto que são arquiteturas multi-processadas, e assim possuem múltiplas instruções para múltiplos dados.

b - Apenas a segunda arquitetura pode ser considerada distribuída, visto que ela possui recursos não compartilhados, isto é, cada CPU possui uma memória local, o que impede conexão entre essas máquinas. Já a primeira arquitetura é multiprocessada, mas todos processadores têm acesso a uma mesma memória compartilhada, o que a desigual, fica como distribuída.

2) O protocolo UDP seria mais adequado. Além de ser mais rápido, o que permite maior agilidade ao lidar com flutuações na temperatura, o protocolo UDP descreve pacotes que não são entregues, coincidindo com o requisito do sistema de não enviar informações desatualizadas. Se o protocolo TCP fosse usado, ele garantiria entrega.

3) O protocolo, por usar dados binários, é mais leve e flexível, porém mais difícil de tratar, o que demanda mais processamento. Já o JSON, por ser um formato textual, ocupa mais espaço e, portanto, pode atrasar a transmissão. No entanto, é facilmente manipulado e compreendido. Assim sendo, protocolo é ideal para casos com menor disponibilidade de comunicação e menor processamento no servidor, enquanto JSON é ideal para redes robustas e computadores com alto desempenho.

conversão de texto é mais custoso
texto é mais fácil de tratar

100134
9 3,3
30
3330400
3000018305
1300
1000
800

4)	núcleos	speedup	speedup-esperado	eficiência
	1	1	1	100% ou 1
	4	3,33	4	83,25% ou 0,8325

Loa

b d

cliente:

```
5)
name("z.n", arquivo);
sock = socket(TCP);
connect(sock, ip, porta);
send(sock, arquivo);
receiveArquivo(sock, arquivo);
close(sock);
```

servidor:

```
#define N 5 #define porta 6000
sock = socket(TCP);
bind(sock, porta);
listen(sock, N);
mr = accept(sock);
int pid = fork();
if(pid == 0) // filho
    receive(mr, arquivo);
    enviarArquivo(mr, arquivo);
    close(mr);
    exit(0); // fecha processo filho
else
    close(sock);
```

6) $p = N/2$

nome = 0

id = rank()

for($i = id$; $i < N$; $i += p$)

nome += amostra[i];

if(id == 0) {

for($i = 1$; $i < p$; $i++$) {

receive(aux, i)

nome += aux;

media = nome / N

for($i = 1$; $i < p$; $i++$)

send(media, i)

} else {

send(nome, 0)

receive(media, 0)

} cont...

cont...

nome = 0

for($i = id$; $i < N$; $i += p$) {

nome += (amostra[i] - media) * (amostra[i] - media)

if(id == 0) {

for($i = 1$; $i < p$; $i++$) {

receive(aux, i)

nome += aux;

variancia = nome / (N-1)

desvio = sqrt(variancia)

} else {

send(nome, 0)

}